Veri Seti Üzerinden Dolandırıcılık Tespiti

Giriş

Veri seti üzerinden dolandırıcılık tespiti için veri madenciliği yapmak istiyorum. Seçtiğim veri seti üzerinden çeşitli algoritmalar uygulayarak bilgi elde etmek ve bu sayede daha başarılı biçimde dolandırıcılık tespiti yapmak hedefimdir.

Makalemin Özeti

Bu çalışmada, veri madenciliği yöntemleri kullanılarak dolandırıcılık tespiti üzerine bir araştırma gerçekleştirilmiştir. Farklı veri madenciliği algoritmalarını deneyerek çeşitli çalışmaları inceleyerek, dolandırıcılık tespitine en uygun algoritmalar ve yollar belirlenmiştir. Araştırma; rastgele ormanlar, destek vektör makineleri (SVM), lojistik regresyon, XGBoost ve k-en yakın komşu (KNN) gibi güçlü algoritmaları değerlendirmiştir.

Çalışma, dolandırıcılık işlemlerini yüksek doğruluk oranlarıyla tespit edebilecek modeller geliştirmeyi ve bu modellerin performansını doğruluk, kesinlik, duyarlılık, F1 skoru (bir modelin kesinlik ve duyarlılık arasında dengeyi ölçen bir metriktir. Dengesiz veri kümelerinde doğruluk değerlendirmesi için önemlidir ve 1'e yaklaştıkça modelin performansı artar) gibi metriklerle değerlendirmeyi amaçlamıştır. Algoritmaların performanslarını karşılaştırarak dolandırıcılık tespitinde kullanılan mevcut veri madenciliği yöntemlerinin doğruluğu ve hızı test edilmiştir. Bu araştırma, dolandırıcılık tespitinde veri madenciliğini kullanarak finansal güvenliği artırmaya yönelik bir araştırma sunmaktadır.

Veri Madenciliği Dersi için İncelenen İlgili Makalelerin Özeti

Adaptive Machine Learning for Credit Card Fraud Detection

Bu tezde geliştirilen dolandırıcılık tespit sistemi, kredi kartı dolandırıcılığını tespit etmek için çeşitli makine öğrenimi algoritmalarını bir araya getirir ve gerçek dünyada kullanılmaya uygun ve yüksek doğruluğa sahip bir çözüm sunmaktadır. Çalışma, sınıf dengesizliği, concept drift ve araştırmacı geri bildirim entegrasyonu gibi veri madenciliği zorluklarını çözmek için HDDT, Rastgele Ormanlar, SVM, Logit Boost, Sinir Ağları ve SMOTE gibi algoritmaları entegre etmektedir. Bu sayede dolandırıcılık tespitinde güvenilir, ölçeklenebilir ve dinamik bir prototip sistem geliştirilmiştir.

Maximum Independent Set Problem İçin Malatya Merkezlilik Algoritması

Bu çalışmada, bir grafın maksimum bağımsız kümesini bulmak için Malatya merkezlilik algoritması önerilmiştir. Maksimum Bağımsız Küme Problemi (MISP), bir grafın düğümleri arasından birbirine bitişik olmayan en fazla sayıda düğüm seçilmesini hedefler ve bu problem NP-zor olarak sınıflandırılır.

Malatya algoritması, maksimum bağımsız küme problemini çözmek için etkili bir yöntem sunar. Algoritmada, önce tüm düğümler için merkezlilik değerleri hesaplanır. Daha sonra, en düşük merkezlilik değerine sahip düğüm seçilerek bağımsız küme setine eklenir. Seçilen düğüm ve komşuları grafikten çıkarılır ve bu işlem tüm düğümler işlenene kadar tekrarlanır. Yapılan analizler, Malatya algoritmasının örnek grafikler üzerinde etkili çözümler sunduğunu ve diğer yöntemlere kıyasla daha verimli olduğunu göstermektedir.

Minimum Vertex Cover Problem İçin Malatya Merkezlilik Algoritması

Bu çalışmada, Minimum Düğüm Kapsama Problemi (MVCP) için Malatya merkezlilik algoritması önerilmiştir. MVCP, bir grafın tüm kenarlarını kapsayacak en az sayıda düğüm seçmeyi hedefler ve NP-zor bir problemdir.

Banka Ödemelerinde Dolandırıcılığın Çizge Madenciliği ve Makine Öğrenimi Algoritmalarıyla Tespiti

Dicle Üniversitesinin hazırladığı "Banka Ödemelerinde Dolandırıcılığın Çizge Madenciliği ve Makine Öğrenimi Algoritmalarıyla Tespiti" makalesi özetle;

Günümüzdeki şirketler önemli verilerini veri tabanlarında saklarlar. Saldırı olduğunda finansal bilgiler hedef alınmaktadır. Bu saldırı türlerinden biri de dolandırıcılık saldırılarıdır. Grafik veri bilimi, dolandırıcılık tespit yöntemlerini daha doğru ve etkili hale getirir. Bu çalışmada İspanya'daki bir banka ödeme bilgi simülasyonundan oluşturulan BankSim veri kümesi kullanılmıştır.

BankSim üzerinde bulunan normal ödemeler ve sahte veriler sınıflandırılarak dolandırıcılık tespiti gerçekleştirilmesi amaçlanmıştır. Sınıflandırma için Python dilinde Rastgele Ormanlar, Destek Vektör Makineleri, XGBoost (XGB), KNN sınıflandırma algoritmaları kullanılmıştır. Performans değerlendirmeleri için K-katlamalı çapraz doğrulama kullanılmıştır. Çizge madenciliği için Neo4j veritabanı kullanılmış ve Neo4j sorgu dili olarak CypherQL kullanılmıştır. Bu dolandırıcılık tespitinin uygulanması sayesinde daha az saldırı işlemleri ve daha güvenli bir akış elde edilmiştir. Çizge madenciliği sırasında PageRank, Community ve derece gibi algoritmalar ile standart makine öğrenimi yöntemleri kullanılarak elde edilen sonuçlar geliştirilmiştir. Bu açıdan çizge madenciliği ve makine öğrenimi algoritmalarının birlikte kullanılmasının diğer yöntemlere kıyasla doğruluk oranlarının daha yüksek olduğu ve daha hızlı sürede hesaplama yapan bir yöntem olduğu ispatlanmıştır.

Denetimli Makine Öğrenmesi Yöntemleri ile Kredi Kartı Sahteciliğini Tahmin Etme: Karşılaştırmalı Analiz

Güner Altan ve Metin Recep Zafer tarafından İstanbul Üniversitesinin İktisat Politikası Araştırmaları Dergisinde yer alan araştırma makalesi özetle;

Dijital platformlar üzerinden yapılan harcamalar günümüzde oldukça yaygınlaşmıştır. Ödeme işlemlerinin bu kadar hızlanması ve artmasından dolayı güvenliği sağlamak zorlaşmıştır. Bankalar, kredi kartı harcamalarının dolandırıcılık olup olmadığını tespit etmekte kilit bir rol üstlenir. Bu çalışmada, bir kamu bankasının 2023 Ocak ayına ait 13.050 kredi kartı işlemi üzerinde yapılan analizle, dolandırıcılığı tespit etmeye yönelik bir model geliştirilmiştir. Python ile yapılan analizde, Lojistik Regresyon, Rastgele Orman, K-En Yakın Komşu, Karar Ağaçları ve Gradyan Güçlendirme algoritmaları kullanılmıştır. Algoritmaların doğruluk oranları %86.4 ile %93.1 arasında değişmiş; kesinlik, duyarlılık, F1 skoru ve ROC-AUC gibi performanslar incelenmiştir. Çalışma, kredi kartı dolandırıcılığını tespitte bu beş algoritmanın iyi birer seçenek olduğunu göstermektedir.

FİNANS SEKTÖRÜNDE DOLANDIRICILIK TESPİTİ ÜZERİNE MELEZ SINIFLANDIRMA VE REGRESYON AĞACI UYGULAMASI

DergiPark ın Yönetim Bilişim Sistemleri Dergisi Cilt:5 Sayı:2 deki bu çalışma, veri madenciliğinin finansal işlemlerde dolandırıcılık tespit etmekte nasıl kullanıldığını ele alıyor. Özellikle, PaySim adlı bir simülatörle oluşturulan sahte bir veri seti üzerinde sınıflandırma yapılmış. Araştırmada, öncelikle klasik CART algoritması kullanılmış, ardından bu algoritma Genetik Algoritma (GA) ile optimize edilerek GA-CART modeli oluşturulmuş. Sonuçlar, GA-CART modelinin klasik CART'a göre %37 daha iyi performans sergilediğini gösteriyor. Bu sayede, finansal illegal işlemlerin tespitinde veri madenciliği tekniklerinin faydalarına dikkat çekilmiştir.

Finansal Tablolarda Hile Riskinin Tespit Edilmesinde Veri Madenciliği Yöntemlerinin Kullanılmasına Yönelik Bir Araştırma

Yaşar Üniversitesi Dergisindeki bu makalede, 2015-2019 yılları arasında Borsa İstanbul'da işlem gören tekstil, giyim eşyası ve deri sektöründeki şirketlerin finansal tablolarındaki hile riskinin veri madenciliği yöntemleriyle tespiti amaçlanmıştır. Çalışmada hileli finansal raporlamayı tespit etmek için 127 finansal tablo incelenmiş ve 12 farklı finansal oran ile veri madenciliği teknikleri kullanılarak analiz yapılmıştır. Bu oranlar arasında brüt kâr/aktif toplamı, çalışma sermayesi/aktif toplamı ve borç/öz sermaye oranı gibi ölçütler bulunur.

Kullanılan veri madenciliği algoritmaları ise Derin Öğrenme, Yapay Sinir Ağı, Karar Ağacı, J48, Rastgele Orman, Destek Vektör Makineleri, K-En Yakın Komşu, AdaBoost, Navie Bayes ve Lojistik Regresyon'dur. Çalışma sonucunda en başarılı yöntemlerin J48 ve Derin Öğrenme olduğu görülmüş, bu iki algoritmanın %84.25 başarı oranıyla en yüksek performansı gösterdiği tespit edilmiştir.

Kaggle - Fraud Transaction Detection

Bu projede, finansal veri seti kullanılarak sahtekar işlemleri tespit etmek için Karar Ağaçları, Rastgele Ormanlar ve KNN veri madenciliği algoritmaları kullanılmıştır. Proje, sahtekar işlemleri tespit etmek için gerekli özellikleri belirlemek, verileri temizlemek ve model oluşturmak gibi adımları içerir.

Karar Ağacı Destekli Hile Tespiti ve Bir Uygulama

Alanya Akademik İncelemeler Dergisindeki bu çalışmada, hileli finansal işlemlerin tespit edilmesi için makine öğrenimi tekniklerinden biri olan karar ağacı algoritması kullanılmıştır. Sertifikalı Hile Denetçileri Birliği'nin (ACFE) hile ağacındaki öne çıkan hileli ödemeler örnek alınarak, özellikle de bankacılık sektöründe hileli işlem riskini azaltmayı amaçlayan bir model geliştirilmiştir. Çalışma, hileli ve normal işlemleri içeren bir yapay veri setiyle gerçekleştirilmiş ve Python kullanılarak uygulamaya geçirilmiştir.

Karar ağacı tekniği, %97.1 doğruluk, %98.4 F1-skor, %98.9 kesinlik ve %98 duyarlılık oranlarıyla oldukça yüksek bir performans sergilemiştir. Finans işlemlerindeki kontrolleri hızlandırıp, şüpheli işlemleri daha çabuk bulmak hedeflenmiştir. Modelin karar ağacı tabanlı yapısı sayesinde yüksek hacimli verilerle çalışabilme, eksik verilere duyarlılığı az olma ve karmaşık veri yapılarında tutarlı sonuçlar verme gibi avantajlara sahiptir.

Çalışmada kullanılan veri madenciliği algoritması olan karar ağacı, veriyi sınıflandırarak hileli işlemlerin tespit edilmesini kolaylaştırır. Yapılan analizlerde, modelin sonuçları incelenmiş ve karar ağacının hileli işlemleri doğru şekilde tahmin edebilme oranının yüksek olduğu görülmüştür.

MAKİNE ÖĞRENİMİ ALGORİTMALARI İLE KREDİ KARTI İŞLEMLERİNDE DOLANDIRICILIK TESPİTİ YÜKSEK LİSANS TEZİ

Bu tezde, kredi kartı işlemlerinde dolandırıcılığı tespit etmek için **Rastgele Orman (Random Forest)**, **Gradyan Artırma Ağaçları (Gradient Boosting Trees)**, **Yapay Sinir Ağları (Artificial Neural Networks)** ve **Karar Ağaçları (Decision Trees)** gibi makine öğrenimi algoritmaları analiz edilmiştir. Amaç, dolandırıcılık riskini öngörmek ve işlemlerin güvenliğini artırmaktır.

Veri seti, gerçek işlemleri yansıtan bir simülasyon modeli ile oluşturulmuş ve algoritmalar şüpheli işlemleri tespit etmeye odaklanmıştır. Karar ağacı tabanlı yöntemlerin daha yüksek doğruluk sağladığı görülmüştür. Çalışma, finansal dolandırıcılığın tespitinde güvenilir ve etkili çözümler sunmaktadır.

Tekdüzen kaynak bulucu yoluyla kimlik avı tespiti için makine öğrenmesi algoritmalarının özellik tabanlı performans karşılaştırması

Bu makalede, makine öğrenimi algoritmaları kullanılarak URL tabanlı kimlik avı (phishing) saldırılarının tespiti incelenmiştir. Amaç, URL adreslerini zararlı veya güvenli olarak sınıflandırarak kimlik avı saldırılarını önlemektir. Destek Vektör Makineleri (SVM), Rastgele Orman, Gaussian Naive Bayes (GNB), Lojistik Regresyon, KNN, Karar Ağaçları, MLP ve XGBoost.algoritmaları kullanılmıştır. Veri seti USOM, Alexa ve PhishTank gibi kaynaklardan 11.935 veri toplanmış, URL uzunluğu, IP adresi içerme ve SSL sertifikası gibi 12 özellik çıkarılmıştır. Sonuç olarak, en yüksek doğruluk %99.8 ile Rastgele Orman algoritmasında elde edilmiştir, GNB algoritması en düşük performansı (%73.4) göstermiştir, Karmaşıklık matrisleri ile doğru ve yanlış sınıflandırmalar analiz edilmiştir.

Çalışma, detaylı özellik çıkarımı ve güncel veri seti kullanımıyla makine öğrenimi yöntemlerinin kimlik avı tespitinde etkili olduğunu göstermiştir.

Veri Seti - Sınıflandırma İlişkisinde Performansa Etki Eden Faktörlerin Değerlendirilmesi

Fırat Üniversitesi Müh. Bil. Dergisindeki bu çalışmada veri madenciliği yöntemlerinden sınıflandırma üzerinde durulmuştur ve sınıflandırmanın başarımına etki eden faktörler detaylı olarak incelenmiştir. Çalışmada k-en yakın komşu , karar ağaçları , rastgele orman , Naive bayes, lojistik regresyon, destek vektör makineleri, gradyan artırma, Adaboost ve yapay sinir ağları kullanılmıştır.

Çalışma, farklı veri setlerinde sınıflandırma algoritmalarının başarısını değerlendirerek test tekniklerinin ve veri seti özelliklerinin performansa etkisini incelemeyi amaçlamaktadır. Literatürde yaygın olarak kullanılan 32 veri seti üzerinde deneyler gerçekleştirilmiş ve kullanılan dokuz farklı sınıflandırma algoritmasıyla başarım kıyaslaması yapılmıştır. Elde edilen bulgular, test yöntemlerinin (Hold-out ve 10 katlı çapraz doğrulama) ve veri seti parametrelerinin sınıflandırma başarımına etkilerini gözler önüne sermiştir. Çalışma, yalnızca doğruluk metriğine bağlı kalmadan F1 ölçütü, Matthews korelasyon katsayısı (MCC) ve alıcı işlem karakteristik eğrisi (AUC) gibi metriklerin önemine dikkat çekerek, doğru sınıflandırıcı modelin seçimi için bir yol haritası sunmuştur.

Kendi düşüncem

Makaleleri inceledim, her biri iyi algoritmalar seçmiş ve bazıları yöntemleri karşılaştırma yapmış. Rastgele Orman (Random Forest), Destek Vektör Makineleri (SVM), Lojistik Regresyon (Logistic Regression), XGBoost (Extreme Gradient Boosting) ve K-En Yakın Komşu (K-Nearest Neighbors, KNN) veri madenciliği algoritmalarının kendi çalışmam için en uygun olabilecek yöntemler olduğuna karar verdim. Sırası ile çalışmam için seçtiğim yöntemler;

Veri Setim Hakkında

Veri seti, **6362620 satır** ve **11 sütun** içermektedir. Bu veriler, her biri 1 saati temsil eden zaman dilimleri üzerinden toplanmıştır ve 30 günlük bir simülasyona dayanmaktadır. Veriler, her bir işlemin özelliklerini ve dolandırıcılık durumu hakkında önemli bilgileri içermektedir.

Veri Setimdeki Sütunlar:

1. **step**: Bu sütun, her bir işlemi temsil eden zaman dilimini belirtir. Her bir adım (step) 1 saatlik bir süreyi ifade eder. Bu sütun toplamda 744 adımı (30 günlük bir simülasyonu) kapsar.
2. **type**: Bu sütun, işlem türünü belirtir. Aşağıdaki türler bulunur:
    - **CASH-IN**: Hesaba para yatırma işlemi
    - **CASH-OUT**: Hesaptan para çekme işlemi
    - **DEBIT**: Hesaptan yapılan ödeme işlemi
    - **PAYMENT**: Ödemeler
    - **TRANSFER**: Para transferleri
3. **amount**: Bu sütun, işlemin yerel para birimindeki tutarını gösterir.
4. **nameOrig**: Bu sütun, işlemi başlatan müşteri veya hesap sahibinin adını belirtir.
5. **oldbalanceOrg**: İşlemden önce, işlem başlatan (sender) hesabın bakiyesi.
6. **newbalanceOrig**: İşlemden sonra, işlem başlatan (sender) hesabın bakiyesi.
7. **nameDest**: Bu sütun, işlemde parayı alan (receiver) müşteri veya hesabın adını belirtir. Ayrıca, bu sütunda "M" ile başlayan müşteri isimleri, ticaret yapan müşteri hesaplarıdır ve bunlara ait bilgiler bulunmamaktadır.
8. **oldbalanceDest**: Alıcı hesabın işlem öncesi bakiyesi. Bu sütun "M" ile başlayan alıcı hesapları için mevcut değildir.
9. **newbalanceDest**: Alıcı hesabın işlem sonrası bakiyesi. "M" ile başlayan hesaplar için bu bilgi de bulunmamaktadır.
10. **isFraud**: Bu sütun, işlemde dolandırıcılık olup olmadığını gösteren ikili bir değişkendir. 1, dolandırıcılık olduğu anlamına gelirken 0, normal bir işlem olduğunu belirtir.
11. **isFlaggedFraud**: Bu sütun, bir işlemde 200.000 veya daha fazla para transferi yapılması durumunda "flag" (işaret) durumunun aktive olduğunu belirtir. Eğer bu değer 1 ise, işlem şüpheli ve potansiyel olarak yasa dışıdır.

Sırasıyla Veri Seti Üzerinde Kullandığım Algoritmalar ve Yapılan İşlemler

Model performansı;

- **Doğruluk (Accuracy)**: Tüm tahminler içinde doğru sınıflandırma oranıdır. Ancak dengesiz veri dağılımlarında yanıltıcı olabilir.
- **Kesinlik (Precision)**: Dolandırıcılık tahminlerinin doğruluğunu gösterir. Yanlış alarmları (False Positive) azaltmada önemlidir.
- **Duyarlılık (Recall)**: Gerçek dolandırıcılık durumlarının tespit başarısını ölçer. Gerçek pozitiflerin (True Positive) kaçırılmaması açısından kritiktir.
- **F1 Skoru**: Kesinlik ve duyarlılık arasındaki dengeyi ölçer.

$$\textbf{F1 Skoru} = 2 \times \left((\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \right)$$

  Bu metrik, dengesiz veri setlerinde modelin genel performansını daha iyi yansıtır.

- **Karışıklık Matrisi (Confusion Matrix)**: Modelin tahminlerini ve gerçek değerleri karşılaştırarak doğru (TP, TN) ve yanlış (FP, FN) sınıflandırmaları gösterir:
    - **TP (True Positive)**: Doğru olarak dolandırıcılık olarak tahmin edilenler.
    - **FP (False Positive)**: Yanlış dolandırıcılık olarak tahmin edilenler.
    - **TN (True Negative)**: Doğru olarak dolandırıcılık değil diye tahmin edilenler.
    - **FN (False Negative)**: Yanlış dolandırıcılık değil diye tahmin edilenler.

İle değerlendirilmiştir.

Rastgele Orman Algoritması

Nedir?

Rastgele Ormanlar (Random Forest), bir dizi bağımsız karar ağacından (decision trees) oluşan ansamble bir öğrenme algoritmasıdır. Her bir karar ağacı, verilerin bir alt kümesi üzerinde eğitim alır ve bu ağaçlardan elde edilen tahminlerin çoğunluk oyu veya ortalama değeri kullanılarak nihai tahmin yapılır. Bu yöntem, özellikle sınıflandırma ve regresyon problemleri için etkili bir çözüm sunar.

Nasıl Çalışır?

Rastgele Ormanlar algoritması, aşağıdaki adımları izleyerek çalışır:

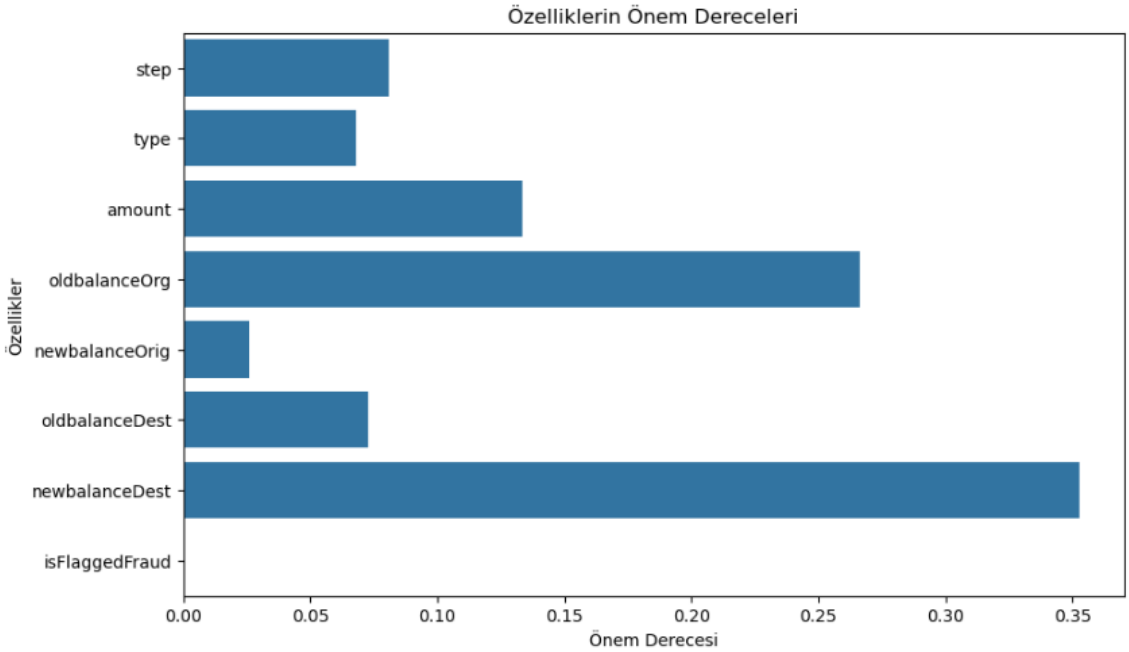
1. **Veri Alt Kümesi Seçimi (Bootstrap)**: Veri setinden rastgele örnekler seçilir. Bu örnekler, her bir karar ağacını eğitmek için kullanılır. Bu işlem, **bootstrap örneklemesi** olarak bilinir.
2. **Karar Ağaçları Oluşturma**: Her ağaç, seçilen alt küme üzerinde eğitim alır. Her ağaçta her düğümde, veri setindeki özelliklerden rastgele bir alt küme seçilir ve en iyi özelliği belirlemek için bu alt küme kullanılır.
3. **Tahmin Yapma**: Yeni bir örnekle karşılaşıldığında, her ağaç tahmin yapar ve bu tahminlerin çoğunluk oyuyla nihai tahmin yapılır (sınıflandırma için) ya da ağaçlardan gelen tahminlerin ortalaması alınır (regresyon için).

Teorik Formül:

1. **Bootstrap Örneklemesi**: Veri setinden $x_1, x_2, \ldots, x_n$ örnekleri rastgele seçilir.
2. **Ağaçlarda Karar Verme**: Her bir düğümde, veri kümesindeki rastgele seçilmiş özellik kümesi üzerinden karar verilir.
3. **Ortalama Tahmin**: Her bir ağaç $f_i(x)$ tahmini yapar ve tahminler birleştirilir:

$$f_{\text{final}}(x) = \frac{1}{T} \sum_{i=1}^{T} f_i(x)$$

Burada T toplam ağaç sayısını ifade eder.



Yukarıdaki grafik Rastgele Ormanlar yöntemini uygularken ağaç oluşunda hangi sütunun ne kadar önemli olduğunu önem derecesi olarak belirterek gösteren grafiktir.

Destek Vektör Makineleri (SVM)

Nedir?

Destek Vektör Makineleri (SVM), denetimli öğrenme algoritmalarından biridir ve genellikle sınıflandırma problemleri için kullanılır. SVM, verileri en iyi şekilde ayıran bir hiper düzlem (hyperplane) bulmayı amaçlar. Bu hiper düzlem, iki sınıf arasındaki sınırı tanımlar ve doğrusal olmayan sınıflandırmalar için de kernel fonksiyonları kullanılarak daha karmaşık sınırlar oluşturulabilir.

Nasıl Çalışır?

1. **Doğrusal Sınıflandırma**:
   - SVM, verileri iki sınıfa ayıran doğrusal bir hiper düzlem arar. Hedef, bu düzlemi veriye en iyi şekilde yerleştirmek ve iki sınıf arasındaki mesafeyi maksimize etmektir.
   - En iyi sınır, sınıfların arasındaki mesafeyi (margin) maksimize eden düzlemdir.
2. **Kernel Fonksiyonları**:
   - Veriler doğrusal olarak ayıramıyorsa, kernel fonksiyonları kullanılarak veriler daha yüksek boyutlu bir uzaya taşınır ve burada doğrusal ayırma yapılır.
   - Yaygın kernel fonksiyonları arasında **lineer**, **polinomial**, **radial basis function (RBF)** ve **sigmoid kernel** bulunur.
3. **Destek Vektörleri**:
   - En önemli veri noktaları, doğru sınıflandırmanın sağlanmasında en etkili olan verilerdir. Bu veri noktalarına **destek vektörleri** denir.
   - Bu vektörler, hiper düzlemin etrafında "destek" sağlar.

Teorik Formül:

Veri seti $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ olsun. Burada x_i veriyi ve y_i ise sınıf etiketini temsil eder.

Hedef, şu doğrusal sınırı bulmaktır:

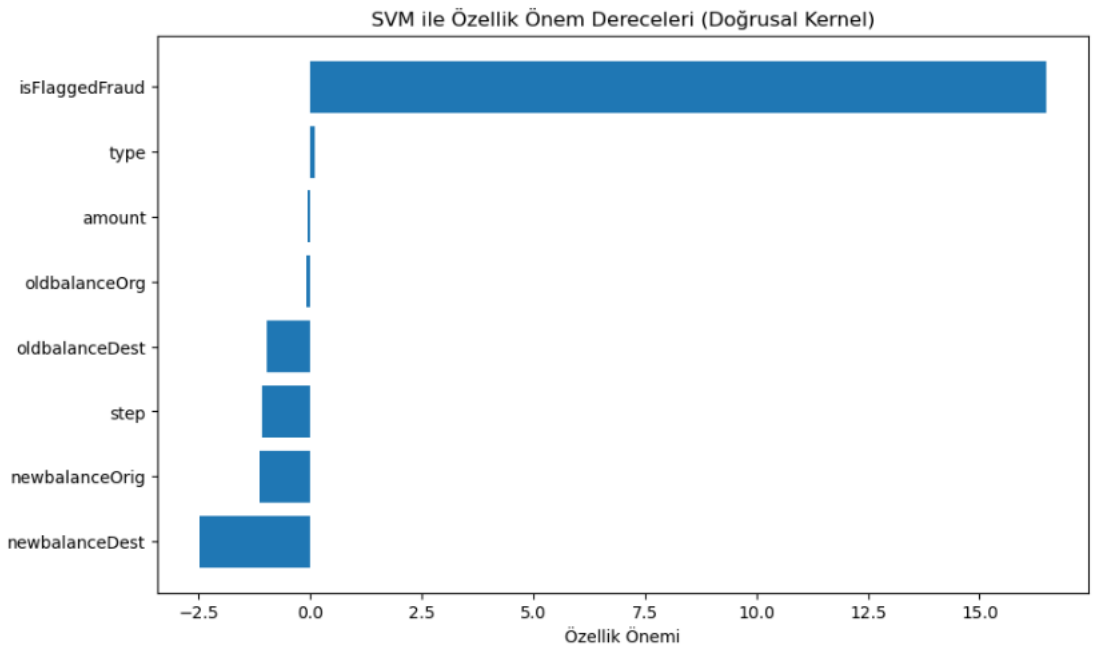
$$w \cdot x + b = 0$$

Burada w normal vektörü ve b bias terimidir. Sınıflandırma, y_i etiketine göre yapılır:

- $y_i = 1$ için, $w \cdot x_i + b \geq 1$
- $y_i = -1$ için, $w \cdot x_i + b \leq -1$

SVM, bu sınıfları ayıran en büyük marjini bulmayı hedefler. Eğer veriler doğrusal olarak ayrılabilir değilse, kernel fonksiyonları kullanılarak veri yüksek boyutlu bir uzaya taşınır.

Fraud Veri Seti ile SVM Uygulaması



Yukarıdaki grafik SVM yöntemini uygularken Destek vektörünü oluştururken hangi sütunun ne kadar önemli olduğunu önem derecesi olarak belirterek gösteren grafiktir.

Lojistik Regresyon (Logistic Regression)

Nedir?

Lojistik regresyon, bir bağımlı değişkenin iki veya daha fazla sınıfa ayrıldığı durumlarda tahmin yapmak için kullanılan denetimli bir makine öğrenimi yöntemidir. Özellikle, ikili sınıflandırma problemlerinde sıkça kullanılır. Lojistik regresyon, lineer regresyonun bir genelleştirilmiş halidir ancak sonuçları sürekli değerler yerine, sınıf olasılıkları olarak üretir.

Nasıl Çalışır?

- **Sigmoid Fonksiyonu:** Lojistik regresyon, girdiler üzerinde bir lineer model oluşturur ve bu lineer modelin çıktısını **sigmoid fonksiyon** üzerinden geçirir. Sigmoid fonksiyonu, sonuçları [0,1] aralığında bir olasılık olarak dönüştürür.
- **Karar Sınırı:** Olasılık değeri bir eşik (genellikle 0.5) ile karşılaştırılır ve sınıf tahmini yapılır.
- **Optimizasyon:** Model, lojistik kayıp fonksiyonunu minimize ederek parametrelerini öğrenir.

Teorik Formül

1. **Sigmoid Fonksiyonu:**

$$h(x) = \frac{1}{1 + e^{-z}}$$

Burada $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$ lineer modeldir.

2. **Tahmin Olasılığı:**

$$P(y = 1|x) = h(x)$$

$$P(y = 0|x) = 1 - h(x)$$

3. **Lojistik Kaybı:**

$$L = -\frac{1}{m} \sum_{i=1}^{m} [y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i))]$$

XGBoost (Extreme Gradient Boosting)

Nedir?

XGBoost (Extreme Gradient Boosting), karar ağaçlarına dayalı bir makine öğrenimi algoritmasıdır. Gradient Boosting yönteminin optimize edilmiş bir sürümüdür ve özellikle büyük veri setlerinde ve yüksek boyutlu veri problemlerinde etkili performansıyla tanınır. Hem sınıflandırma hem de regresyon problemlerinde kullanılabilir. XGBoost, hız, doğruluk ve esneklik açısından avantajlar sağlar.

Nasıl Çalışır?

XGBoost, birden çok zayıf öğreniciyi (genellikle karar ağaçları) ardışık olarak birleştirerek güçlü bir tahmin modeli oluşturur. Her adımda, önceki modelin hatalarını azaltmaya odaklanır. Bu işlem, gradyan iniş yöntemi ile gerçekleştirilir.

Ana Adımlar:

1. Başlangıçta, tahminler rastgele yapılır veya bir sabit değerden başlar.
2. İlk modelin yaptığı hatalar, bir kayıp fonksiyonu (genellikle MSE) kullanılarak hesaplanır.
3. Bu hataları azaltmak için yeni bir zayıf öğrenici eğitilir.
4. Tüm modellerin çıktıları birleştirilir.

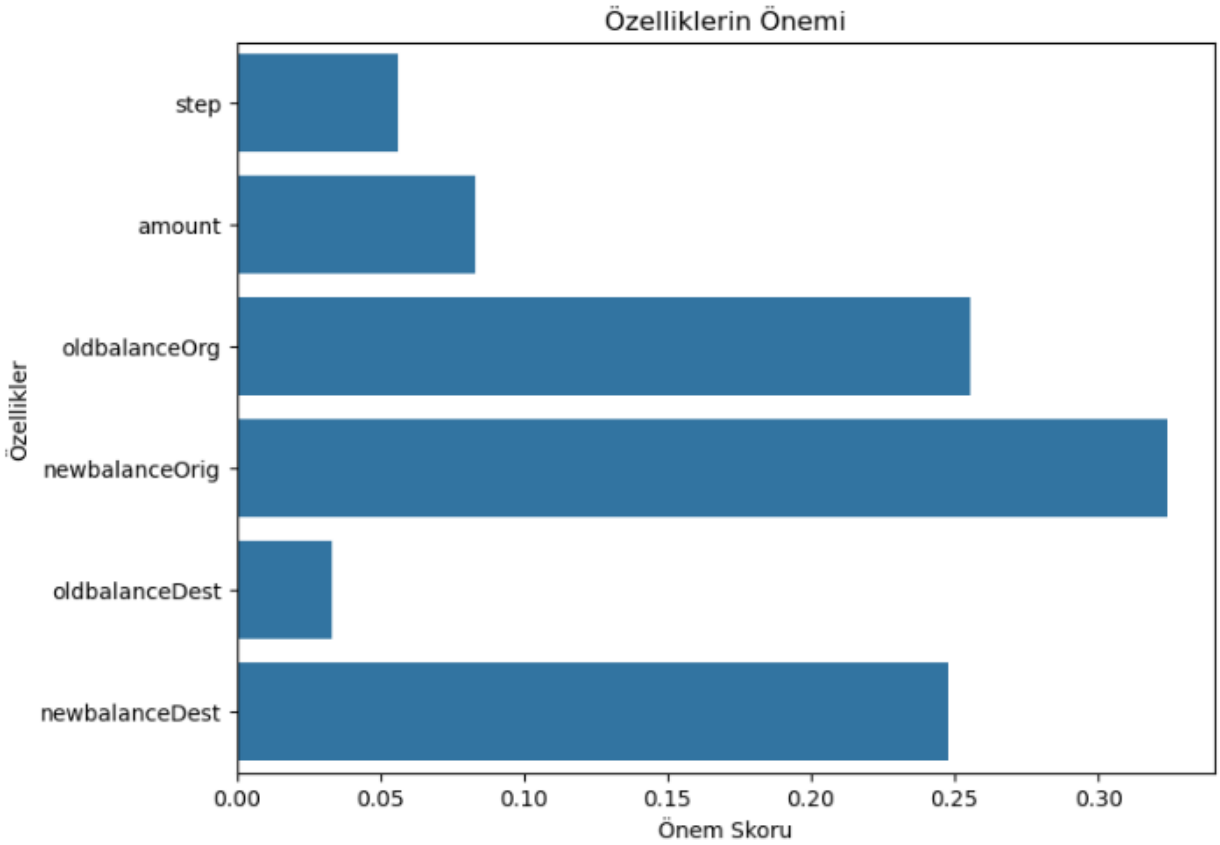
Teorik Formül: Kayıp fonksiyonu aşağıdaki gibi ifade edilir:

$$L(t) = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

- ℓ: Hata fonksiyonu (örneğin, log-loss veya MSE).
- $\hat{y}_i^{(t-1)}$: $t-1$ adımındaki tahmin.
- $f_t(x_i)$: t-inci zayıf öğrenicinin çıktısı.
- $\Omega(f_t)$: Modelin karmaşıklık cezası.

XGBoost, ayrıca model karmaşıklığını dengeleyen $\Omega(f_t)$ terimi ile aşırı öğrenmeyi kontrol eder.

XGBoost Kodlama Örneği



Yandaki grafik XGBoost yöntemini uygularken hangi sütunun ne kadar önemli olduğunu önem derecesi olarak belirterek gösteren grafiktir.

K-En Yakın Komşu (K-Nearest Neighbors, KNN)

Nedir?

K-En Yakın Komşu (KNN), denetimli öğrenme kategorisinde yer alan basit ve etkili bir makine öğrenimi algoritmasıdır. Hem sınıflandırma hem de regresyon problemleri için kullanılabilir. KNN, yeni bir veri noktasının sınıfını tahmin etmek için, bu noktaya en yakın kkk veri noktasını (komşularını) analiz eder ve çoğunluğa göre karar verir.

Nasıl Çalışır?

1. Yeni bir veri noktası için en yakın **k** komşu seçilir.
2. Komşuluk, genellikle **Öklid Uzaklığı** veya **Manhattan Uzaklığı** gibi bir mesafe metriğiyle hesaplanır.
3. Sınıflandırma için: Komşular arasında en sık görülen sınıf tahmin edilir (çoğunluk oylaması).
   Regresyon için: Komşuların ortalama değeri kullanılır.
4. **k** parametresi, algoritmanın esnekliğini ve doğruluğunu etkiler.
   - Çok küçük **k**: Model gürültüye duyarlı olabilir.
   - Çok büyük **k**: Model fazla genelleştirilebilir.

Teorik Formül

Öklid Uzaklığı: İki nokta (x1,y1) ve (x2,y2) arasındaki mesafe:

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- d: Uzaklık değeri
- n: Özellik sayısı

Komşular arasındaki sınıf dağılımına göre karar:

$$\text{Sınıf}(x) = \arg\max_{c \in C} \sum_{i \in N} \delta(y_i, c)$$

- δ(yi,c): Komşunun sınıfının ccc olup olmadığını kontrol eder.
- N: k en yakın komşu kümesi
- C: Tüm sınıfların kümesi

Malatya Merkezlilik

Malatya merkezlilik, bir grafın düğümleri arasındaki ilişkilerin etkinliğini ölçen bir algoritmadır. Bu algoritma, graf teorisindeki Minimum Vertex Cover (MVCP) ve Maximum Independent Set (MISP) gibi NP-zor problemleri çözmek için optimize edilmiştir.

Malatya merkezlilik değeri ($\psi(v)$), bir düğümün derecesini, komşu düğümlerinin derecelerine göre normalize ederek hesaplar. Bir düğümün merkezlilik değeri şu formülle ifade edilir:

$$\psi(v_i) = \sum_{v_j \in N(v_i)} \frac{d(v_i)}{d(v_j)}$$

Burada:

- v_i: Grafın bir düğümü.
- $N(v_i)$: v_i düğümünün komşuları.
- $d(v_i)$: v_i düğümünün derecesi (komşu düğüm sayısı).

MVCP:
En yüksek merkezlilik değerine sahip düğümler, grafın tüm kenarlarını kapsayacak şekilde seçilir.

MISP:
En düşük merkezlilik değerine sahip düğümler, birbirine bitişik olmayacak şekilde seçilir.

Örnek Veri Seti ve Çözüm

Veri Seti:
Dolandırıcılık tespiti için bir grafik kullanalım. Grafik şu düğümleri ve kenarları içeriyor:

- **Düğümler:** [1, 2, 3, 4, 5, 6]
- **Kenarlar:** [(1, 2), (1, 3), (2, 4), (3, 5), (4, 6), (5, 6)]

Adım 1: Malatya Merkezlilik Hesaplama

Düğüm	Derece ($d(v)$)	Komşuları ($N(v)$)	Merkezlilik ($\psi(v)$)
1	2	[2, 3]	$2/3 + 2/3 = 1.33$
2	2	[1, 4]	$2/2 + 2/1 = 3.0$
3	2	[1, 5]	$2/2 + 2/1 = 3.0$
4	2	[2, 6]	$2/2 + 2/2 = 2.0$
5	2	[3, 6]	$2/2 + 2/2 = 2.0$
6	2	[4, 5]	$2/2 + 2/2 = 2.0$

Adım 2: Minimum Vertex Cover (MVCP)

1. **En yüksek merkezlilik değeri:** $\psi(2) = 3.0$. Düğüm 2 seçilir ve grafdan çıkarılır.
2. Güncellenen graf üzerinden aynı işlemi tekrarlayarak çözüm kümesi belirlenir: $\{2, 3, 6\}$.

Adım 3: Maximum Independent Set (MISP)

1. **En düşük merkezlilik değeri:** $\psi(1) = 1.33$. Düğüm 1 seçilir ve komşuları çıkarılır.
2. Bu adımları tekrarlayarak bağımsız küme bulunur: $\{1, 4, 6\}$.

Malatya Merkezlilik Hesaplama Kod Örneği

Minimum Düğüm Kapsama (MVCP) kümesini belirleme kodu

```
def find_min_vertex_cover(graph):
    """Minimum Düğüm Kapsama (MVCP) kümesini belirler."""
    vertex_cover = set()
    while graph.number_of_edges() > 0:
        centrality_values = malatya_centrality(graph)
        max_node = max(centrality_values, key=centrality_values.get)
        vertex_cover.add(max_node)
        graph.remove_node(max_node)  # Düğümü ve bağlantılarını kaldır
    return vertex_cover
```

Maksimum Bağımsız Küme (MISP) kümesini belirleme kodu

```python
def find_max_independent_set(graph):
    """Maksimum Bağımsız Küme (MISP) kümesini belirler."""
    independent_set = set()
    while graph.number_of_edges() > 0:
        centrality_values = malatya_centrality(graph)
        min_node = min(centrality_values, key=centrality_values.get)
        independent_set.add(min_node)
        neighbors = list(graph.neighbors(min_node))
        graph.remove_node(min_node)  # Düğümü kaldır
        graph.remove_nodes_from(neighbors)  # Komşularını kaldır
    return independent_set
```

Malatya Merkezlilik Hesaplama kodu

```python
def malatya_centrality(graph):
    """Graf düğümlerinin Malatya merkezlilik değerlerini hesaplar."""
    centrality_values = {}
    for node in graph.nodes:
        neighbors = list(graph.neighbors(node))
        degree_node = graph.degree[node]
        centrality = sum(degree_node / graph.degree[neighbor] for neighbor in neighbors)
        centrality_values[node] = centrality
    return centrality_values
```

Karşılaştırma

Yöntem	Doğruluk	Kesinlik	Duyarlılık	F1 Skoru
Rastgele Ormanlar	0.9996	0.9939	0.7111	0.8291
Destek Vektör Makineleri (SVM)	0.9992	0.9548	0.3784	0.5420
Lojistik Regresyon	1.0000	0.9000	0.4800	0.6300
XGBoost	0.9996	0.9591	0.6951	0.8060
K-En Yakın Komşu (KNN)	0.9989	0.7500	0.2056	0.3227
Malatya Merkezlilik	0.4300	0.3100	0.9800	0.4720

1. **Rastgele Ormanlar (Random Forests)**:
    - Genel olarak en başarılı yöntemlerden biri. F1 skoru (%82.91) oldukça yüksek, kesinlik (%99.39) ve doğruluk (%99.96) da çok iyi seviyede. Ancak duyarlılığı (%71.11) biraz daha artırılabilir.
2. **Destek Vektör Makineleri (SVM)**:
    - Kesinlik oranı (%95.48) fena değil ama duyarlılık (%37.84) ve F1 skoru (%54.20) düşük kaldı. Bu, modelin sınıf dengesizliğiyle iyi başa çıkamadığını gösteriyor.
3. **Lojistik Regresyon**:
    - Doğruluk oranı %100 gibi görünse de bu, büyük ihtimalle dengesiz veri setinin bir etkisi. F1 skoru (%63.00) ve duyarlılık (%48.00) daha iyi olabilir. Yine de basit ve hızlı bir model olarak iş görebilir.

4. **XGBoost**:
    - Performansı dengeli ve istikrarlı. F1 skoru (%80.60) ile Rastgele Ormanlar'a yakın, kesinlik (%95.91) ve duyarlılık (%69.51) ise iyi seviyede. Güçlü bir alternatif.
5. **K-En Yakın Komşu (KNN)**:
    - Bu yöntem veri setiniz için pek uygun değil gibi görünüyor. Kesinlik (%75.00) düşük, duyarlılık (%20.56) çok düşük. F1 skoru da (%32.27) vasat.
6. **Malatya Merkezlilik**:
    - Bu yöntemin duyarlılığı oldukça yüksek (%98.00), ancak kesinlik (%31.00) ve F1 skoru (%47.20) düşük seviyede. Duyarlılığın kritik olduğu durumlarda kullanılabilir, ancak dengeli performans gerektiren senaryolarda yetersiz kalabilir.

Sonuç ve Değerlendirmem

Rastgele Ormanlar ya da XGBoost kullanmak mantıklı görünüyor, çünkü hem duyarlılık hem de kesinlik arasında iyi bir denge sağlıyorlar. Verilerdeki dengesizliği düzeltmek için SMOTE gibi yöntemlerle "Fraud" artırabilirsiniz. Diğer modellerdeki düşük performansı iyileştirmek için hiperparametre optimizasyonu yapabilirsiniz. Genel olarak model seçimi, ihtiyacınıza göre duyarlılık mı yoksa kesinlik mi öncelikli olduğuna bağlı. Ama ben XGBoost yöntemini hem hızından hem de dengeli olduğu için veri setim için en iyi yöntem olduğunu düşünüyorum.

Kaynakça

1. **Adaptive Machine Learning for Credit Card Fraud Detection**
   Université Libre de Bruxelles, Bilgisayar Bilimi Makine Öğrenimi Grubu. Uyarlanabilir makine öğrenimi algoritmalarını kullanarak dolandırıcılık tespitine yönelik zorlukları ele alan bir tez.
2. **Banka Ödemelerinde Dolandırıcılığın Çizge Madenciliği ve Makine Öğrenimi Algoritmalarıyla Tespiti**
   Dicle Üniversitesi, BankSim veri seti üzerinde dolandırıcılık tespiti için grafik veri bilimi ve makine öğrenimi yöntemlerini inceleyen çalışma.
3. **Denetimli Makine Öğrenmesi Yöntemleri ile Kredi Kartı Sahteciliğini Tahmin Etme: Karşılaştırmalı Analiz**
   Altan G. ve Zafer M.R., İstanbul Üniversitesi. Kredi kartı dolandırıcılığı tespiti üzerine algoritmaların karşılaştırmasını içeren araştırma.
4. **Finans Sektöründe Dolandırıcılık Tespiti Üzerine Melez Sınıflandırma ve Regresyon Ağacı Uygulaması**
   Yönetim Bilişim Sistemleri Dergisi, Genetik Algoritmalar ile optimize edilmiş sınıflandırma modelleri.
5. **Finansal Tablolarda Hile Riskinin Tespit Edilmesinde Veri Madenciliği Yöntemlerinin Kullanılması**
   Yaşar Üniversitesi, 2015-2019 yıllarında hileli finansal raporlamayı inceleyen veri madenciliği çalışması.
6. **Kaggle - Fraud Transaction Detection**
   Karar Ağaçları, Rastgele Ormanlar ve KNN algoritmalarını kullanarak finansal dolandırıcılık tespit çalışması.
7. **Karar Ağacı Destekli Hile Tespiti ve Bir Uygulama**
   Alanya Akademik İncelemeler Dergisi, karar ağacı tabanlı finansal hile tespitine yönelik model geliştirme.
8. **Makine Öğrenimi Algoritmaları ile Kredi Kartı İşlemlerinde Dolandırıcılık Tespiti**
   Hitit Üniversitesi, kredi kartı işlemlerini analiz eden yüksek lisans tezi.
9. **Tekdüzen Kaynak Bulucu Yoluyla Kimlik Avı Tespiti için Makine Öğrenmesi Algoritmalarının Performans Karşılaştırması**
   URL tabanlı kimlik avı saldırılarını sınıflandırmaya yönelik makine öğrenimi analizi.
10. **Veri Seti - Sınıflandırma İlişkisinde Performansa Etki Eden Faktörlerin Değerlendirilmesi**
    Fırat Üniversitesi, veri sınıflandırmasında algoritma başarılarını karşılaştıran çalışma.
11. https://dergipark.org.tr/tr/download/article-file/2734758 A New Approach Based on Centrality Value in Solving the Minimum Vertex Cover Problem: Malatya Centrality Algorithm
12. https://dergipark.org.tr/en/download/article-file/2853600 A New Approach Based on Centrality Value in Solving the Maximum Independent Set Problem: Malatya Centrality Algorithm